

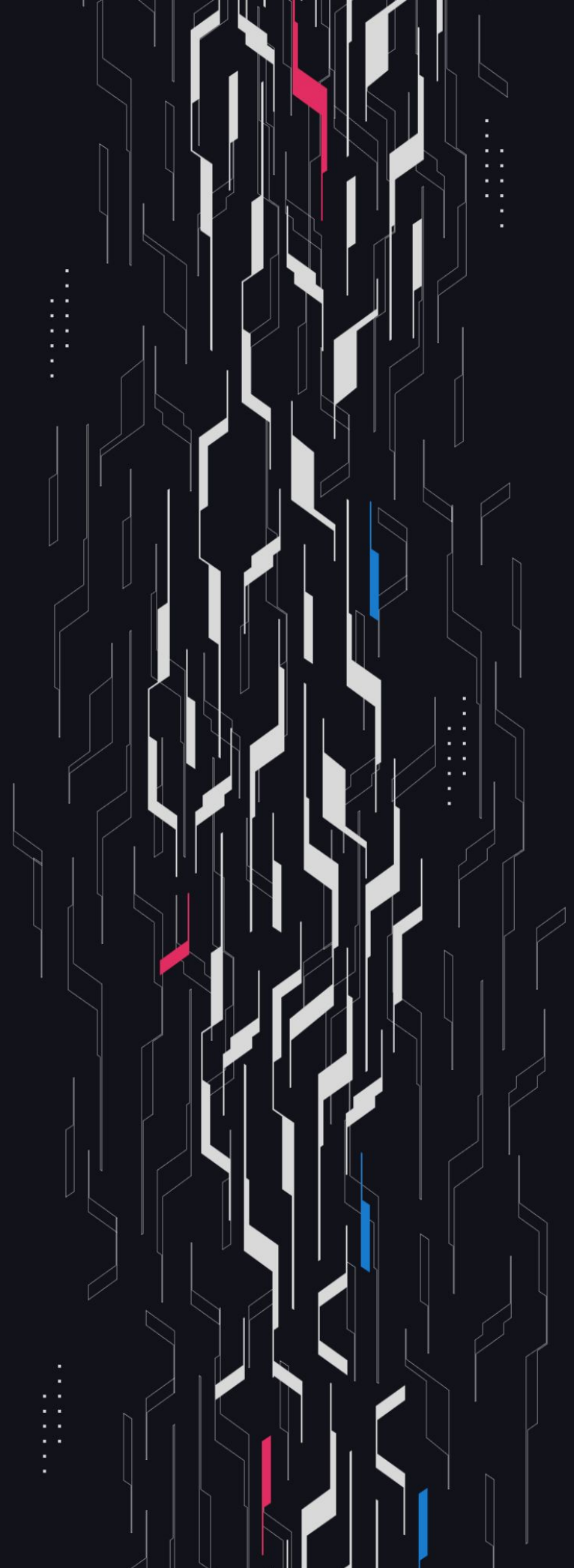
GA GUARDIAN

Degen Safe

Contract Updates

Security Assessment

March 18th, 2026



Summary

Audit Firm Guardian

Prepared By Osman Ozdemir, Ali Shehab

Client Firm Degen Safe

Final Report Date 03/18/2026

Audit Summary

Degen Safe engaged Guardian to review the security of its Contract Updates. From the 03/09/2026 to the 03/12/2026, a team of 2 auditors reviewed the source code in scope. All findings have been recorded in the following report.

Confidence Ranking

Given the lack of High and Critical issues detected during the main review, Guardian assigns a Confidence Ranking of 5 to the protocol. Guardian advises the protocol to consider periodic review with future changes. For detailed understanding of the Guardian Confidence Ranking, please see the rubric on the following page.

Note: Fixes to the findings uncovered in the remediation review have not been reviewed by Guardian.

✓ Verify the authenticity of this report on Guardian's GitHub: <https://github.com/guardianaudits>

📊 Code coverage & PoC test suite:

Guardian Confidence Ranking

Confidence Ranking	Definition and Recommendation	Risk Profile
5: Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4: High Confidence	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. Recommendation: Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3: Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas (e.g., weak modularity, risky patterns). No critical design flaws, though some patterns could lead to issues in edge cases. Recommendation: Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2: Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. Recommendation: Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week. Or additional High/Critical findings uncovered in remediation review which have not been resolved and confirmed by Guardian.
1: Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥ 5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. Recommendation: Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥ 5 High/Critical findings and overall systemic flaws.

Table of Contents

Project Information

Project Overview 5

Audit Scope & Methodology 6

Smart Contract Risk Assessment

Findings & Resolutions 9

Addendum

Disclaimer 28

About Guardian 29

Project Overview

Project Summary

Project Name	Degen Safe
Language	Solidity
Codebase	https://github.com/VasilVR/degensafe-programs
Commit(s)	Main Review Commit: f526ae9b3f078c765e3d35711b5015d985221a0a Remediation Review Commit: 6a4296b0e7c0d3d3a17383d6dc56a8458e0049e8

Audit Summary

Delivery Date	03/18/2026
Audit Methodology	Static Analysis, Manual Review, Test Suite, Contract Fuzzing

Vulnerability Summary

Vulnerability Level	Total	Pending	Declined	Acknowledged	Partially Resolved	Resolved
● Critical	0	0	0	0	0	0
● High	0	0	0	0	0	0
● Medium	1	0	0	0	0	1
● Low	5	0	0	0	0	5
● Info	11	0	0	3	1	7

Audit Scope & Methodology

Diff:
<https://github.com/VasilVR/degensafe-programs/compare/e5c87bc2a753dad699f8421c54c0d8ee4cf72e83...f526ae9b3f078c765e3d35711b5015d985221a0a>

File	blank	comment	code
stake/programs/stake_program/src/lib.rs	214	278	1055
spl-token-vault/programs/ spl_token_vault_program/src/lib.rs	103	82	462
sol-vault/programs/sol_vault_program/src/lib.rs	84	94	359
SUM:	401	454	1876

Audit Scope & Methodology

Vulnerability Classifications

Severity	Impact: <i>High</i>	Impact: <i>Medium</i>	Impact: <i>Low</i>
Likelihood: <i>High</i>	● Critical	● High	● Medium
Likelihood: <i>Medium</i>	● High	● Medium	● Low
Likelihood: <i>Low</i>	● Medium	● Low	● Low

Impact

- High** Significant loss of assets in the protocol, significant harm to a group of users, or a core functionality of the protocol is disrupted.
- Medium** A small amount of funds can be lost or ancillary functionality of the protocol is affected. The user or protocol may experience reduced or delayed receipt of intended funds.
- Low** Can lead to any unexpected behavior with some of the protocol's functionalities that is notable but does not meet the criteria for a higher severity.

Likelihood

- High** The attack is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount gained or the disruption to the protocol.
- Medium** An attack vector that is only possible in uncommon cases or requires a large amount of capital to exercise relative to the amount gained or the disruption to the protocol.
- Low** Unlikely to ever occur in production.

Audit Scope & Methodology

Methodology

Guardian is the ultimate standard for Smart Contract security. An engagement with Guardian entails the following:

- Two competing teams of Guardian security researchers performing an independent review.
- A dedicated fuzzing engineer to construct a comprehensive stateful fuzzing suite for the project.
- An engagement lead security researcher coordinating the 2 teams, performing their own analysis, relaying findings to the client, and orchestrating the testing/verification efforts.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross-referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.
Comprehensive written tests as a part of a code coverage testing suite.
- Contract fuzzing for increased attack resilience.

Findings & Resolutions

ID	Title	Category	Severity	Status
M-01	Changing Reward Token Causes Loss Of Rewards	Logical Error	● Medium	Resolved
L-01	Locked Rewards In Old Vault After Mint Update	Validation	● Low	Resolved
L-02	No Duplicate Check In Update_reward_percentage	Validation	● Low	Resolved
L-03	Rent On User PDAs Not Reclaimable On Withdraw	Validation	● Low	Resolved
L-04	Reward Mint Check Bypassed On Update	Logical Error	● Low	Resolved
I-01	Notes Regarding Fee-On-Transfer Support	Informational	● Info	Resolved
I-02	Unused Errors	Error	● Info	Resolved
I-03	No Guard On Reward Deposit Or Withdrawal	Rewards	● Info	Partially Resolved
I-04	Deprecated Rent Sysvar Passed As Account	Best Practices	● Info	Resolved
I-05	Missing Events	Events	● Info	Resolved
I-06	No Cancel Path For Deposits And Trust Assumptions	Trust Assumptions	● Info	Acknowledged
I-07	Consider Using is_on_curve Check For Updates	Informational	● Info	Acknowledged
I-08	Warning Related To Offchain Part	Warning	● Info	Acknowledged

M-01 | Changing Reward Token Causes Loss Of Rewards

Category	Severity	Location	Status
Logical Error	● Medium	lib.rs: 291-294	Resolved

Description

Checking `total_staked == 0` before allowing a reward mint swap, if nobody has staked, should be safe. The problem is that `total_staked` only tracks active deposits. It ignores `user_stake.unclaimed`.

A user can withdraw their full stake while the reward vault is empty. When that happens, the code stores all owed rewards into `user_stake.unclaimed` and decrements `pool.total_staked` to 0. The pool now passes the guard, but there's still a user owed rewards in the old mint.

Once the admin swaps the mint, `claim_reward` pays from the new vault. The user's `unclaimed`, denominated in the old token, gets paid in the new token. If the tokens have different decimals or value, the user either loses money or gets overpaid.

Example: User has 50 USDC (6 decimals) unclaimed. Admin swaps to a 9-decimal token. User claims and gets `50_000_000` raw units of the new token = 0.05 tokens instead of 50.

Recommendation

The guard needs to also check that no user has outstanding unclaimed rewards.

Resolution

Degen Safe: Resolved.

L-01 | Locked Rewards In Old Vault After Mint Update

Category	Severity	Location	Status
Validation	● Low	lib.rs: 885-893	Resolved

Description

update_reward_mint allows the admin to update the reward token, and this instruction updates the reward_vaultt based on the new token.

The admin can withdraw reward tokens from the reward vault. However, withdrawals are only supported for the current reward vault. If the reward mint is updated while some rewards from the previous token remain unclaimed, there is no mechanism to withdraw those remaining tokens from the previous reward vault.

Recommendation

Consider allowing a reward mint update only after all rewards from the previous mint have been claimed and/or withdrawn. Alternatively, allow the admin to withdraw tokens from previous reward_vaults.

Resolution

Degen Safe: Resolved.

L-02 | No Duplicate Check In Update_reward_percentage

Category	Severity	Location	Status
Validation	● Low	lib.rs: 315	Resolved

Description

The `update_reward_percentage` instruction does not check whether the new reward rate is equal to the current reward rate. As a result, the reward rate can be updated to the same value. While this does not affect reward calculations, it unnecessarily creates an additional epoch, which is subject to a maximum limit.

Recommendation

Add a check in the instruction to ensure the new reward rate differs from the current rate.

Resolution

Degen Safe: Resolved.

L-03 | Rent On User PDAs Not Reclaimable On Withdraw

Category	Severity	Location	Status
Validation	● Low	spl_token_vault_program/src/lib.rs, sol_vault_program/src/lib.rs, stake_program/src/lib.rs	Resolved

Description

For each of the below accounts:

- stake_program: user_stake PDA (DepositStake, init_if_needed, payer = user).
- spl_token_vault_program: deposit_record PDA (Deposit, init, payer = user).
- sol_vault_program: deposit_record PDA (Deposit, init, payer = depositor).

the user (or depositor) pays rent at creation. The programs never close these accounts: there is no close = ... constraint and no instruction that closes them. So after a user fully withdraws stake (stake program) or the deposit record is no longer needed (vault programs), the rent lamports remain locked in the PDA forever. Users cannot reclaim that rent, and the protocol does not redirect it to any beneficiary.

Recommendation

Consider adding an instruction that closes the account when it is safe, or documenting that rent is non-recoverable so users and integrators can account for it.

Resolution

Degen Safe: Resolved.

L-04 | Reward Mint Check Bypassed On Update

Category	Severity	Location	Status
Logical Error	● Low	lib.rs: 155-158	Resolved

Description

At pool creation, `create_pool` enforces that the reward mint must match the staking token mint:

```
require!(
  ctx.accounts.token_mint.key() == ctx.accounts.reward_mint.key(),
  CustomError::RewardMintMustMatchStakeMint
);
```

The comment above it (line 158) explains the rationale:

"Enforce same-token staking: reward mint must be the same as the staking token mint.
This eliminates decimal mismatch issues and simplifies reward calculations."

However, the `update_reward_mint` function performs no such check.

Recommendation

Consider either allowing to have any token, or remove the `update_reward_mint` function.

Resolution

Degen Safe: Resolved.

I-01 | Notes Regarding Fee-On-Transfer Support

Category	Severity	Location	Status
Informational	● Info	lib.rs: 537-542	Resolved

Description

The deposit flow in `spl-token-vault` records the actual amount of tokens received to support fee-on-transfer tokens. In contrast, the deposit flow in the staking program does not perform this check and accepts the provided amount without verification. As a result, the staking program does not support fee-on-transfer tokens in its current form.

This creates a discrepancy between the `spl-token-vault` and staking programs regarding support for fee-on-transfer tokens.

Additionally, the current programs do not support Token-2022 and only interact with the legacy SPL Token program. Since the legacy token program does not implement hooks or fee-on-transfer logic, the before/after balance check becomes redundant when using legacy tokens, even though the comments state fee-on-transfer token support.

Recommendation

Document this behavior if it is intended. Alternatively, consider aligning the behavior between the programs. If fee-on-transfer token support is necessary, consider using Token-2022.

Resolution

Degen Safe: Resolved.

I-02 | Unused Errors

Category	Severity	Location	Status
Error	● Info	stake_program,spl_token_vault	Resolved

Description

Six error variants are defined but never referenced anywhere in the code.

stake_program ([lib.rs](#))

DecimalMismatch

InvalidProgramData

InitializeConfig

spl_token_vault ([lib.rs](#))

Unauthorized

NotRentExempt

InvalidDataLength

CorruptedTokenAccount

Recommendation

Consider removing all unused errors.

Resolution

Degen Safe: Resolved.

I-03 | No Guard On Reward Deposit Or Withdrawal

Category	Severity	Location	Status
Rewards	● Info	stake_program/src/lib.rs	Partially Resolved

Description

`withdraw_reward` lets the admin pull any amount from the reward vault with no cap. There's nothing stopping them from withdrawing tokens that are already owed to stakers, the function doesn't track how much of the vault is committed.

On the flip side, there's also no requirement for the admin to deposit enough rewards to cover what stakers have accrued. If the vault is underfunded, `claim_reward` reverts and `withdraw_stake` silently pays 0 rewards.

Both are admin responsibility, but the program doesn't enforce either.

Recommendation

Track committed rewards at pool level and cap admin withdrawals to the uncommitted balance or consider documenting this behavior for users.

Resolution

Degen Safe: Partially Resolved.

I-04 | Deprecated Rent Sysvar Passed As Account

Category	Severity	Location	Status
Best Practices	● Info	stake_program/src/lib.rs , spl_token_vault_program/src/lib.rs	Resolved

Description

Sysvar<'info, Rent> is deprecated by Solana. The runtime still supports it but the recommended approach is Rent::get().

sol-vault already does this, but stake and spl-token-vault still pass it as an account.

Recommendation

Remove pub rent: Sysvar<'info, Rent> from all account structs and use Rent::get().

Resolution

Degen Safe: Resolved.

I-05 | Missing Events

Category	Severity	Location	Status
Events	● Info	stake_program/src/lib.rs	Resolved

Description

Three state-changing functions only use `msg!()` logs but don't emit typed events:

- `initialize_config` – sets the global admin
- `transfer_admin` – transfers global admin to a new address
- `update_pool_authority` – changes pool owner

All other state-changing functions (`create_pool`, `deposit_stake`, `withdraw_stake`, `claim_reward`, etc.) properly emit typed events. These three are the only ones that don't.

Recommendation

Add typed events for all three, e.g. `ConfigInitializedEvent`, `AdminTransferredEvent`, `PoolAuthorityUpdatedEvent`.

Resolution

Degen Safe: Resolved.

I-06 | No Cancel Path For Deposits And Trust Assumptions

Category	Severity	Location	Status
Trust Assumptions	● Info	Global	Acknowledged

Description

Both `sol-vault` and `spl-token-vault` expose only a `deposit(order_id, amount)` instruction for users. Deposited funds move into PDAs fully controlled by the program, and the only withdrawal instruction can be invoked by the vault authority to sweep all funds to a preconfigured wallet. No user-facing cancel/withdraw path exists, and deposit records are read-only receipts.

Users must rely entirely on the operator’s off-chain infrastructure to acknowledge deposits, fulfill services, or process refunds. A failure or malicious action off-chain leaves deposits permanently locked or lost upon authority withdrawal.

Recommendation

This issue is intended to inform users, and no change is required as this is a design choice. Consider documenting these behaviors and expected workflows.

Resolution

Degen Safe: Acknowledged.

I-07 | Consider Using is_on_curve Check For Updates

Category	Severity	Location	Status
Informational	● Info	Global	Acknowledged

Description

set_withdrawal_account, update_authority, transfer_admin, and update_pool_authority instructions only deny a small set of obviously invalid pubkeys and do not verify that the new authority/account is compatible with the program’s wallet-based authorization model. Since these roles are expected to be controlled by ordinary signer wallets, assigning an off-curve address such as a PDA, or another non-signable address class, can render future authority-gated operations unusable.

While it is impossible to check and deny all possible unusable addresses, it is recommended to use Pubkey::is_on_curve() to at least ensure the address corresponds to a valid Ed25519 point.

Recommendation

Consider validating new addresses using Pubkey::is_on_curve() when updating addresses or wallets.

Resolution

Degen Safe: Acknowledged.

I-08 | Warning Related To Offchain Part

Category	Severity	Location	Status
Warning	● Info	Global	Acknowledged

Description

Users are expected to interact with the protocol via the backend, and `order_ids` are provided by the backend during deposits. However, users can deposit directly on-chain and provide arbitrary `order_ids`. In such cases, there is no guarantee that these deposits will be accounted for or refunded manually. Users must be made aware of this behavior and warned accordingly. They should not interact with the protocol directly on-chain.

Additionally, backend validation must account for arbitrarily supplied on-chain `order_ids` and ensure it does not reuse an already-used `order_id` if IDs are generated via simple incrementation.

Recommendation

Inform and warn users not to perform direct deposits, document the protocol's expected behavior, and ensure that backend validation accounts for this case.

Resolution

Degen Safe: Acknowledged.

Remediation Findings & Resolutions

ID	Title	Category	Severity	Status
L-01	Incorrect Committed_rewards Accounting Allows Draining Of Rewards	Logical Error	● Low	Resolved
I-01	Misplaced Comment	Informational	● Info	Resolved
I-02	Notes Related To Committed Rewards	Informational	● Info	Resolved
I-03	Warning Regarding Deposit Record Closure	Warning	● Info	Resolved

L-01 | Incorrect Committed_rewards Accounting Allows Draining Of Rewards

Category	Severity	Location	Status
Logical Error	● Low	lib.rs: 655	Resolved

Description

In `withdraw_stake`, when rewards get paid out (`reward_to_send > 0`), both `prev_unclaimed` and `pending` are subtracted from `pool.committed_rewards`:

```
pool.committed_rewards = pool.committed_rewards
    .saturating_sub(prev_unclaimed)
    .saturating_sub(pending);
```

The problem is that `pending`, freshly calculated by `calculate_pending_reward`, was never added to `committed_rewards` to begin with. `committed_rewards` only grows when a user interaction materializes rewards (via `deposit_stake`, or the `else` branch of `withdraw_stake` when rewards stay as unclaimed). So subtracting `pending` here removes value that was never accounted for, deflating the counter.

Since `saturating_sub` clamps to 0 instead of reverting, the transaction goes through without error. The pool's accounting just gets quietly corrupted.

```
let uncommitted = reward_vault.amount.saturating_sub(pool.committed_rewards);
```

Every time a user withdraws stake and gets rewards paid, `committed_rewards` drops further than it should. `uncommitted` grows, and the admin can pull out tokens that are owed to other stakers. After enough withdrawals the counter can hit 0, leaving the entire vault exposed.

Recommendation

Only subtract `prev_unclaimed` when rewards are paid out, `pending` was never added so it should not be subtracted.

Resolution

Degen Safe: Resolved.

I-01 | Misplaced Comment

Category	Severity	Location	Status
Informational	● Info	lib.rs: 315-316	Resolved

Description

After adding the `close_deposit_record` instruction in the `spl-token-vault`, the comment related to the `update_authority` instruction remained above the newly added instruction.

```
/// Transfer vault authority to a new address.  
/// Validates the new authority is not a reserved address.  
/// Close a deposit record and return rent to the depositor.  
/// Only the original depositor can close their own record.  
pub fn close_deposit_record(_ctx: Context<CloseDepositRecord>, _order_id: String) ->  
Result<()> {
```

Recommendation

Update the comment.

Resolution

Degen Safe: Resolved.

I-02 | Notes Related To Committed Rewards

Category	Severity	Location	Status
Informational	● Info	stake_program/src/lib.rs	Resolved

Description

The `committed_rewards` field is introduced to track rewards earned by users and to prevent the admin from withdrawing them. However, it only accounts for "unclaimed" rewards and does not include "pending" rewards.

If a user stakes and remains idle for a long time, their rewards continue to accrue as "pending", but are not recorded as "unclaimed" until they interact with their stake (e.g., deposit, withdraw, or claim). As a result, these rewards are not included in `committed_rewards`

It is not feasible to track all users' pending rewards across the codebase, as doing so would require iterating over all users or a significant architectural change. However, it is important to note that pending rewards are not protected and can still be withdrawn.

Recommendation

Document this behavior as a design choice and inform users that pending rewards may be withdrawn. Stakers should periodically update their stake or claim if they want their rewards to be protected.

Optionally, introduce a guard such as: 'if `pool.total_staked > 0`, only allow withdrawals above a defined safety buffer.'

Resolution

Degen Safe: Resolved.

I-03 | Warning Regarding Deposit Record Closure

Category	Severity	Location	Status
Warning	● Info	lib.rs: 319	Resolved

Description

Users can close their deposit records to reclaim the rent. The `CloseDepositRecord` instruction ensures that only the original depositor can close the record.

However, there are no additional restrictions on when a deposit record may be closed. A user may close it even before the backend or off-chain components of the protocol have processed or fulfilled that deposit. Once closed, the on-chain deposit record is deleted and can no longer be queried through `check_deposit`.

Users should be clearly warned not to close a deposit record before the backend has fully processed it, as doing so removes the on-chain record and may interfere with verification or reconciliation of that deposit.

Recommendation

Document this behavior and warn users about the deposit record closures.

Resolution

Degen Safe: Resolved.

Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian’s position is that each company and individual are responsible for their own due diligence and continuous security. Guardian’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

To learn more, visit <https://guardianaudits.com>

To view our audit portfolio, visit <https://github.com/guardianaudits>

To book an audit, message <https://t.me/guardianaudits>